Learning Embeddings for a Pursuit–Evasion Differential Game

1 Game setup

We consider a two-player pursuit–evasion differential game between an *evader* E and a *pursuer* P in the plane. Each agent's velocity is decomposed into a (fixed) magnitude and a unit direction:

$$\vec{v} \;=\; \|\vec{v}\| \cdot \hat{u}, \qquad \hat{u} \in \mathbb{S}^1. \tag{1}$$

The speeds are fixed hyperparameters of the game,

$$v_E \;=\; \text{fixed}, \qquad v_P \;=\; \text{fixed}, \tag{2}$$

and, without loss of generality, the pursuer starts at the origin:

$$\vec{x}_P(0) \;=\; (0,\,0). \tag{3}$$

The state of the game at time t is the pair of positions $\big(\vec{x}_E(t),\,\vec{x}_P(t)\big) \in \mathbb{R}^2 \times \mathbb{R}^2$, evolving on the interval $t \in [0, t_{\max}]$ according to

$$\dot{\vec{x}}_E(t) \;=\; \vec{v}_E(t), \qquad \dot{\vec{x}}_P(t) \;=\; \vec{v}_P(t). \tag{4}$$

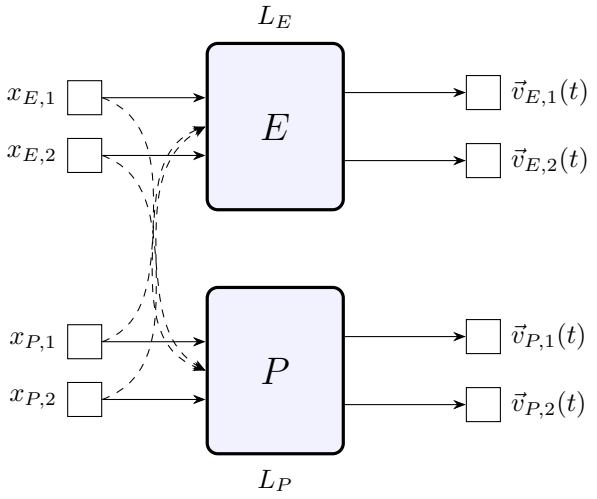
2 Neural network embeddings

Each agent has its own neural network that, at every time t, embeds the current game state into a *unit direction* of motion. Writing the network outputs as $\hat{u}_E^{NN}$ and $\hat{u}_P^{NN}$, the actual velocities used to advance the dynamics are obtained by rescaling these embeddings by the (fixed) speeds:

$$\vec{v}_E(t) \;=\; \|\vec{v}_E\| \cdot \hat{u}_E^{NN}\big(\vec{x}_E(t),\,\vec{x}_P(t)\big), \tag{5}$$

$$\vec{v}_P(t) \;=\; \|\vec{v}_P\| \cdot \hat{u}_P^{NN}\big(\vec{x}_E(t),\,\vec{x}_P(t)\big). \tag{6}$$

The two-network architecture (one per player) is depicted below. The inputs $x_{E,1}, x_{E,2}$ are the components of the evader's position and $x_{P,1}, x_{P,2}$ are the components of the pursuer's position; the outputs $\vec{v}_{E,1}(t), \vec{v}_{E,2}(t)$ and $\vec{v}_{P,1}(t), \vec{v}_{P,2}(t)$ are the components of the resulting velocity vectors.



Solid arrows: an agent's own positions feed its network. Dashed arrows: the opponent's positions are also provided as input, since each policy depends on the relative configuration of the two players.

A more compact view of the same architecture treats each network as a black box producing the player's velocity:

$$\big(\vec{x}_E,\, \vec{v}_E^{NN},\, \vec{v}_P^{NN}\big) \;\longmapsto\; \big(\vec{v}_E(t),\, \vec{v}_P(t)\big). \tag{7}$$

3 Loss functions

Let $\varepsilon > 0$ denote the capture radius and let $t_{\max}$ be the time horizon of a rollout. Define the capture condition

$$\mathcal{C} \;:\; \exists T \le t_{\max} \text{ such that } \|\vec{x}_E(T) - \vec{x}_P(T)\| \le \varepsilon, \qquad t \in [0, t_{\max}]. \tag{8}$$

Case 1 – capture occurs at time T. The pursuer is rewarded for catching the evader as quickly as possible, while the evader is penalised for being caught early:

$$L_E \;\propto\; \frac{1}{T}, \qquad L_P \;\propto\; T. \tag{9}$$

Case 2 – no capture within the horizon. If $\mathcal{C}$ does not hold, the loss falls back to the final separation distance:

$$L_E \;\propto\; \frac{1}{\|\vec{x}_E - \vec{x}_P\|}, \qquad L_P \;\propto\; \|\vec{x}_E - \vec{x}_P\|. \tag{10}$$

Summarising as a piecewise objective,

$$L_E \;=\; \begin{cases} \dfrac{1}{T} & \text{if } \mathcal{C} \text{ holds at time } T, \\[2ex] \dfrac{1}{\|\vec{x}_E(t_{\max}) - \vec{x}_P(t_{\max})\|} & \text{otherwise,} \end{cases} \qquad L_P \;=\; \begin{cases} T & \text{if } \mathcal{C} \text{ holds at time } T \\ \|\vec{x}_E(t_{\max}) - \vec{x}_P(t_{\max})\| & \text{otherwise.} \end{cases} \tag{11}$$

The two networks are trained adversarially: gradients of L_E update the evader's parameters, gradients of L_P update the pursuer's. The unit vectors $\hat{u}_E^{NN}, \hat{u}_P^{NN}$ produced by the networks act as *embeddings* of the current game state into the optimal direction of motion under each player's objective.